

Reto AI-First · Fase 1 — Documento interno (NO COMPARTIR)

Marco de evaluación y notas de facilitación

Uso exclusivo: Diego, Javier, Juan David. No incluir en el material que se comparte por Drive.

Objetivo real del ejercicio

Más allá del entregable, este reto busca **detectar quién se adapta al nuevo modelo operativo AI-first** y alimentar las decisiones de reorganización de perfiles tras el "momento cero" (fin del código manual). Es insumo para decisiones de muy corto plazo.

Alcance — quién participa

- **Incluidos (Ronda 1):** desarrolladores (integraciones, mejoras SILIN, etc.). Detienen su trabajo actual durante el reto.
- **Excluidos por ahora:** cloud / infraestructura (abordaje distinto, se evalúa aparte).
- **Track QA / pruebas: activo en paralelo**, con su propio brief. Trabajan sobre una **aplicación base provista** y construyen la estrategia de pruebas con IA (no idean proyecto propio).
- **Excepción — Marisleidy Mora:** continúa con el módulo de pagos manuales ya liberado, dándole continuidad hacia producción con el mismo enfoque AI-first. No inventa proyecto nuevo.

Por qué Hermes es obligatorio (para nosotros)

Hermes es el punto de control: captura contexto, interacciones y evolución. Es lo que nos permite medir *cuánto* y *cómo* trabajaron, no solo el resultado final. El `HERMES_CONTEXT.md` de cada participante alimenta el Hermes global de evaluación.

Rúbrica de evaluación

Dimensión	Qué observamos	Peso
Entregable Hermes (MD)	Claridad, trazabilidad del proceso	25%
Autonomía / resolución	Investigó, desbloqueó solo, curiosidad	25%
Orquestación de IA	Calidad de specs, iteración, manejo de contexto	18%
Funcionalidad end-to-end	Todos los componentes presentes y funcionando	14%
Previsión y comunicación	Levantó la mano a tiempo (créditos, bloqueos)	10%
Criterio técnico	Decisiones de arquitectura razonables	8%

Nota: las dos dimensiones de mayor peso —el entregable Hermes y la autonomía— son las más predictivas del nuevo modelo: reflejan la capacidad de conducir el proceso por cuenta propia y dejar un rastro de contexto claro. Es coherente con el hallazgo del experimento Hermes: la profundidad y disciplina del contexto pesan más que el modelo elegido.

Rúbrica — Track QA

Misma filosofía, ajustada a pruebas (sustituye "funcionalidad" por "cobertura/profundidad"):

Dimensión	Qué observamos	Peso
Entregable Hermes (MD)	Claridad, trazabilidad del proceso	25%
Autonomía / resolución	Investigó, desbloqueó solo, curiosidad	25%
Orquestación de IA	Calidad de specs, iteración, manejo de contexto	18%
Cobertura y profundidad de pruebas	Funcional, datos, integración, automatización	14%
Previsión y comunicación	Levantó la mano a tiempo (créditos, bloqueos)	10%
Criterio de QA	Estrategia, priorización por riesgo, definición de "done"	8%

Aplicación base (SUT) para QA — mini-tienda-base

App full-stack autocontenida (FastAPI + SQLite + frontend JS), entregada como `.zip` junto al brief de QA. Corre con `docker compose up` o `uvicorn app:app`. Incluye frontend, API

REST, BD SQLite y una integración de precios/impuestos con simulación de caída (`PRICING_FAIL=1`).

Ground truth — defectos sembrados (NO incluido en el material de QA)

Para que el reporte de defectos sea significativo, la app contiene tres debilidades sutiles e intencionales que **no rompen la ejecución**. Sirven de "respuestas" para evaluar la profundidad de las pruebas:

#	Defecto	Cómo se manifiesta
1	Sin validación de cantidad	<code>qty</code> 0 o negativa se acepta; con negativa el total queda negativo y el stock sube
2	Truncamiento de impuesto	El impuesto usa <code>int()</code> (trunca) en vez de redondear; p. ej. 10% sobre 1005 da 100 en vez de 101
3	País sensible a mayúsculas	<code>country</code> en minúscula (<code>co</code>) cae a la tasa por defecto (10%) en vez de 19%

Escenario adicional esperado (no es defecto): con `PRICING_FAIL=1` , la cotización y la creación de orden deben responder **503** — buen caso para probar el manejo de la caída de la integración.

Un QA fuerte debería detectar al menos los defectos 1 y 3 sin pistas y cubrir el escenario de caída de la integración. Si prefieres un SUT "limpio", basta con quitar esas tres líneas del código.

Formato de evaluación — "cata por grupos"

- Día 5: demos cortas (5-7 min) + revisión cruzada por grupos.
- Modalidad acordada: construcción **individual**, revisión en **grupos**.

Logística de lanzamiento

- **Lunes:** Juan + Javier abren y enmarcan la expectativa ("los estamos preparando para el futuro"); Diego toma el control operativo del reto.
- **Distribución del brief:** archivo compartido en Google Drive (PDF o doc).

Decisiones pendientes antes del lunes

1. Modalidad: individual vs grupos → **individual + revisión en grupo** (cerrado).
2. Tema libre vs menú de ideas-semilla → **libre** (cerrado).

3. ~~Track de QA/pruebas~~ → **track activo en paralelo, con brief propio** (cerrado).
4. **Entregar la aplicación base** para el track QA (backend + frontend + BD + web service + integración).
5. Afinar la ruta de aprendizaje (validar recursos).
6. Fechas/cronograma definitivos.

Estado de documentos

- **Brief DEV:** aprobado (versión final).
- **Brief QA:** borrador v0.1, pendiente de validación con Juan David y Javier.